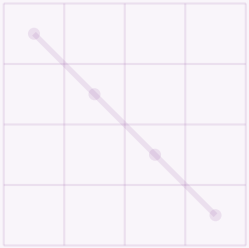
11

Лекция 20–21 апреля

Проблема остановки

Константин Чухарев

Дискретная математика — Весна 2027



Диагональный аргумент

> *Сущность математики лежит именно в её свободе.*
>
> ***— Георг Кантор***

Диагональ Кантора

Диагональный метод Кантора доказывает несчётность действительных чисел.

Дан список чисел $r_1, r_2, r_3, \ldots$ — строим новое число, меняя n-ю цифру на отличную от n-й цифры r_n.

Новое число отличается от каждого элемента списка в своей позиции — списка всех действительных чисел не существует.

> Тот же приём убьёт мечту об универсальном анализаторе программ.
>
> Диагонализация везде устроена одинаково: объект, отличающийся от любого элемента перечисления, обязан в нём состоять.

Инверсия диагонали

Все машины выстраиваются в список $M_1, M_2, M_3, \ldots$: каждая задаётся конечным текстом.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$
M_1	ост.	ост.	цикл	ост.
M_2	цикл	цикл	ост.	ост.
M_3	ост.	цикл	цикл	цикл
M_4	ост.	ост.	ост.	цикл
D — инверсия	цикл	ост.	ост.	ост.

Строка D — инверсия диагонали: на коде $\langle M_i \rangle$ она делает противоположное M_i.

Но D тоже машина — на своей позиции списка она обязана отличаться от себя.

Самоприменимость машин

Кодировка $\langle M \rangle$ — такая же строка, как любая другая, а машина умеет читать строки.

Подать машине её собственное описание — не страннее, чем передать функции её номер.

После универсальной машины это уже не сюрприз: программа читает программу штатно.

Диагонализация требует лишь одного — выразительности, достаточной для разговора о себе.

Проблема остановки

> *Математику можно определить как предмет, в котором мы никогда не знаем ни того, о чём говорим, ни того, истинно ли то, что мы говорим.*
>
> ***— Бертран Рассел***

Язык HALT

Определение 1 («HALT»)

По паре $(\langle M \rangle, w)$ узнать, останавливается ли M на входе w:

$$\text{HALT} = \{\langle M \rangle w \mid M \text{ останавливается на } w\}.$$

Пример (Конкретные запросы)

- M_{loop} — на любом входе сразу зацикливается: $\langle M_{\text{loop}} \rangle 0 \notin \text{HALT}$.
- M_{acc} — на любом входе сразу принимает: $\langle M_{\text{acc}} \rangle \varepsilon \in \text{HALT}$.

Тривиальные случаи решаются локально — вопрос в одном универсальном алгоритме для всех пар.

Почему симуляция не решает?

Наивный план: запустить U на $\langle M \rangle w$ и подождать ответа.

Если M останавливается — симуляция увидит это.

Если M зациклилась, ждать можно вечно.

Любой конечный таймаут произволен: зациклившаяся машина может остановиться на секунду позже.

Симулятор лишь воспроизводит проблему — нужен другой решатель, и его не существует.

Неразрешимость остановки

Теорема 1

Не существует МТ, которая по паре $(\langle M \rangle, w)$ всегда останавливается и решает, остановится ли M на w.

Доказательство — от противного, в четыре шага:

1. допустить, что решатель H существует
2. собрать машину-обёртку D над текстом H
3. подставить D в неё саму
4. получить противоречие разбором случаев

Шаг 1: гипотетический решатель

Определение 2 (Решатель HALT)

МТ H называется решателем языка HALT, если на входе $\langle M \rangle w$ она

- всегда останавливается
- принимает, когда M останавливается на w
- отвергает, когда M зацикливается на w

Допущение выглядит скромно: H — обычная машина, чья программа есть конечный текст, а не магический оракул.

Конечный текст можно скопировать и встроить в другую машину как подпрограмму — на этом и поедет всё доказательство.

Шаг 2: машина-обёртка

Пример (Программа, построенная из программы)

Имея текст H, собираем машину D:

1. прочитать вход — описание $\langle M \rangle$
2. составить строку $\langle M \rangle \langle M \rangle$
3. вызвать H на этой строке
4. при приёме H — уйти в бесконечный цикл
5. при отвержении H — остановиться

D — законная машина Тьюринга: встраивание готовой машины как подпрограммы мы уже делали при строительстве универсальной машины.

Смысл обёртки — инверсия: на вопрос о паре $(\langle M \rangle, \langle M \rangle)$ машина D отвечает противоположно решателю H.

Шаг 3: подстановка машины в себя

Машина D готова работать с любым описанием на входе — подадим ей собственное, $\langle D \rangle$.

На входе $\langle D \rangle$ верны сразу два утверждения, каждое по своему определению.

Контракт решателя на паре $(\langle D \rangle, \langle D \rangle)$:

$$H(\langle D \rangle \langle D \rangle) \text{ принимает} \leftrightarrow D \text{ останавливается на } \langle D \rangle$$

Код обёртки на входе $\langle D \rangle$:

$$D \text{ останавливается на } \langle D \rangle \leftrightarrow H(\langle D \rangle \langle D \rangle) \text{ отвергает}$$

Собираем цепочку: приём H равносилен остановке D, а остановка D — отвержению H.

Шаг 4: противоречие

Доказательство

Докажем разбором случаев поведение D на входе $\langle D \rangle$.

Случай 1: D останавливается на $\langle D \rangle$. Тогда по контракту H на паре $(\langle D \rangle, \langle D \rangle)$ принимает. По коду обёртки приём H переводит D в бесконечный цикл — вопреки остановке.

Случай 2: D зацикливается на $\langle D \rangle$. Тогда H на той же паре отвергает. По коду обёртки отвержение заставляет D остановиться — вопреки циклу.

Любое поведение D опровергает контракт H, поэтому решателя HALT не существует.

Где именно ломается допущение

Противоречие бьёт не по машине D, а по предположению о решателе.

Машина не обязана останавливаться

Противоречия внутри D нет: машина имеет право зацикливаться, обязанность всегда останавливаться была только у H. Ломается допущение: при существующем H машина D обязана на $\langle D \rangle$ и остановиться, и зациклиться.

Распознаваема, но не разрешима

Распознаваемость HALT растёт из симуляции: универсальная машина повторяет работу M на w и принимает при остановке.

Неразрешимость выросла из диагонали: любой решатель ломается машиной D на собственном коде.

Дополнение $\overline{\text{HALT}}$ — пары, на которых M зацикливается — даже не распознаваемо.

Будь оно распознаваемым, из двух распознавателей собрался бы разрешитель HALT — вопреки теореме.

Распознаваемость с двух сторон

Вторую сторону гарантии даёт зеркальный распознаватель.

Теорема 2 (Критерий разрешимости)

Язык L разрешим тогда и только тогда, когда распознаваемы оба языка: L и $\overline{L}$.

Доказательство

Докажем эквивалентность в обе стороны.

Если L разрешим, разрешитель распознаёт L, а та же машина с инвертированным ответом распознаёт $\overline{L}$.

Обратно: распознаватели R и S запускаются синхронно, по одному шагу за такт.

Слово лежит ровно в одном из двух языков, поэтому рано или поздно примет одна из машин, и её ответ будет верным.

Замыкание распознаваемых языков

Распознаватель обещает приём своих слов и ничего больше.

Теорема 3

Распознаваемые языки замкнуты относительно объединения и пересечения, но не дополнения.

Доказательство

Объединение. Распознаватели работают синхронно: составной принимает, как только приняла любая из машин.

Пересечение. Последовательный запуск: сначала R_1, затем R_2. Слово вне пересечения не примет ни одна из двух — и это законно.

Дополнение. Замкнутости нет: иначе критерий разрешимости дал бы разрешитель HALT.

Замыкание разрешимых языков

Разрешители останавливаются на любом входе, поэтому операции собираются без оговорок.

Теорема 4

Разрешимые языки замкнуты относительно объединения, пересечения и дополнения.

Доказательство

Объединение. Разрешители идут по очереди: принимаем, если принял хотя бы один.

Пересечение. Тот же последовательный запуск, принимаем, если приняли оба.

Дополнение. Инвертируем ответ — остановка гарантирована.

Разница классов — в дополнении.

У разрешимых машины работают по обе стороны черты, у распознаваемых — с одной.

Одно семейство доказательств

Объект	Самореференция	Полученное «нет»
Лжец	$S \leftrightarrow \neg S$	нет истинностного значения
Рассел	$R = \{x \mid x \notin x\}$	нет множества
Кантор	диагональ вне списка	нет перечисления
Гёдель	«недоказуемо»	истина вне вывода
Тьюринг	D на своём коде	нет алгоритма

Система, достаточно выразительная, чтобы говорить о себе, порождает неразрешимость.

Неразрешимость — не наша неспособность придумать алгоритм, а свойство выразительности.

Сведения

> *Если задачу решить не удаётся, рядом есть более простая задача, которую решить можно. Найдите её.*
>
> ***— Дьёрдь Пойа***

Сведение

Определение 3 (Сведение (*many-one reduction*))

$A \underset{m}{\leq} B$, если существует вычислимая тотальная функция f, для которой

$$w \in A \leftrightarrow f(w) \in B.$$

Решив B, решаем и A: считаем $f(w)$ и спрашиваем решатель B.

Сведение переносит алгоритмы — и неразрешимости.

Как пользоваться сведением?

Теорема 5 (Логика переноса)

- Если $A \leq_m B$ и B разрешима, то разрешима и A.
- Контрапозитивно: если $A \leq_m B$ и A неразрешима, то неразрешима и B.

Доказательство

Пусть f — функция сведения, R — разрешитель B.

На входе w машина вычисляет $f(w)$, запускает R и выдаёт его ответ — по свойству сведения это ответ на вопрос $w \in A$.

Оба шага останавливаются на каждом входе, поэтому A разрешима, а контрапозиция даёт второй пункт.

Направление сведения

Сведение через контрапозицию превращается в инструмент доказательства неразрешимости. Направление выбирается один раз: ошибка направления обесценивает всё построение. Каждый пример ниже следует этому правилу — от HALT к новой задаче.

Сводить A к разрешимой B бессмысленно — из этого следует лишь разрешимость A.

Чтобы доказать неразрешимость задачи X, сводят к ней известную неразрешимую: $\mathrm{HALT} \underset{m}{\leq} X$.

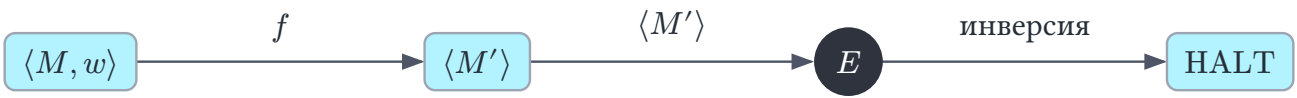
Пример: пустота языка

Пример (От HALT к E_{TM})

$E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \varnothing\}$ — описания машин с пустым языком.

Построение $f(\langle M \rangle, w) = \langle M' \rangle$: M' игнорирует свой вход, симулирует M на w и принимает при остановке.

M остановилась — язык Σ^*, зациклилась — язык $\varnothing$.



Разрешитель E_{TM} с инверсией ответа решал бы HALT — значит, E_{TM} неразрешима.

Остановка на пустом входе

Пример («HALT» $\underset{m}{\leq}$ «HALT» на ε)

Язык: $\mathrm{HALT}_\varepsilon = \{\langle M\rangle \mid M \text{ останавливается на пустом входе}\}$.

Обёртка $f(\langle M\rangle, w) = \langle M'\rangle$ — машина M' на любом входе делает так:

1. стирает ленту и записывает на неё строку w
2. возвращает головку к началу записи
3. дальше исполняет таблицу переходов машины M

На пустом входе M' сама готовит себе строку w — дальше машины неотличимы.

Тогда M останавливается на $w \leftrightarrow M'$ останавливается на ε.

Сведение: эквивалентность программ

Пример («HALT» $\underset{m}{\leq} \mathrm{EQ}_{\mathrm{TM}}$)

Язык: $\mathrm{EQ}_{\mathrm{TM}} = \{\langle M_1 \rangle \langle M_2 \rangle \mid L(M_1) = L(M_2)\}$ — пары машин с равными языками.

Обёртка $f(\langle M \rangle, w) = \langle M' \rangle \langle A \rangle$ из двух машин:

- M' игнорирует свой вход, симулирует M на w и принимает, если симуляция остановилась
- A принимает любой вход сразу

M остановилась на $w \Rightarrow L(M') = \Sigma^* = L(A)$ — машины эквивалентны.

M зациклилась $\Rightarrow L(M') = \varnothing \neq L(A)$ — машины различны.

Ответ «да» решателя $\mathrm{EQ}_{\mathrm{TM}}$ — сразу ответ «да» для HALT, инверсия не нужна.

Сведение: «печатает 42»

Пример («HALT» $\leq_m \mathrm{PRINT}_{42}$)

Язык: $\mathrm{PRINT}_{42} = \{\langle P\rangle \mid P$ когда-нибудь печатает $42\}$.

Обёртка $f(\langle M\rangle, w) = \langle P\rangle$ — программа P на пустом входе делает так:

1. симулирует M на входе w
2. если симуляция остановилась — печатает 42
3. пока симуляция идёт — ничего не печатает

P напечатает 42 тогда и только тогда, когда M остановится на w.

Три сведения — один приём: обёртка вшивает вычисление M на w, а свойство задачи подгоняется под наблюдаемое поведение обёртки.

Меняется только линза: пустота языка, равенство языков, появление 42 в выводе.

Программа анализирует программу

Антивирус хочет решить: опасна ли эта программа?

Статический анализатор хочет решить: содержит ли код утечку?

Верификатор хочет решить: удовлетворяет ли программа спецификации?

Всё это — свойства языка, который распознаёт программа, и все сводятся от HALT.

Почему точные решатели здесь невозможны в принципе — теорема Райса.

Верификация работала в разрешимых фрагментах — теперь видна цена этой оговорки.

Другие неразрешимые задачи

Пример (Семейство от HALT)

- **Тотальность**: останавливается ли M на каждом входе?
- **Эквивалентность**: верно ли $L(M_1) = L(M_2)$?
- **Регулярность**: является ли $L(M)$ регулярным?
- **Контекстная свобода**: является ли $L(M)$ контекстно-свободным?

Каждая задача — сведение от HALT со своей вспомогательной машиной.

Все спрашивают не об устройстве машины, а о свойствах её языка.

Что остаётся инженеру?

Неразрешимость запрещает не инструменты, а обещание полноты: идеального отладчика и универсального доказателя завершения не будет.

Каждый практический инструмент спасается одним из трёх ходов:

1. сузить класс программ, о которых спрашивают
2. оставить инструменту право ответить «не знаю»
3. заменить программу моделью и отвечать о модели

Выбор хода — инженерное решение с известной ценой: чем шире класс, тем грубее ответ.

Инструменты и их честная рамка

Инструмент	Обещание	Честная рамка
Отладчик	найти любое зависание	один прогон — вечный цикл выглядит как долгая работа
Анализатор завершения	доказать завершение	правила для узких классов циклов — вне класса «не знаю»
Линтер	поймать все дефекты	заданные паттерны — срабатывания и пропуски неизбежны
Система типов	принять всё корректное	разрешимый фрагмент — отвергает часть корректных
Проверка моделей (*model checking*)	проверить сам код	проверяет конечную модель, не сам код

Что мы поняли?

Диагональ Кантора переехала на программы: полный список машин с ответами противоречив. Из него выросли неразрешимость остановки и сведение как рабочий инструмент. Дополнение разделяет классы: у распознаваемых языков оно не замыкается.

- Диагональный аргумент: инвертированная диагональ не влезает в перечисление машин.
- HALT распознаваема симуляцией, но неразрешима: D ломает любой решатель.
- Сведение переносит неразрешимость: пустота, тотальность, эквивалентность, регулярность.
- Разрешимость — распознаваемость с двух сторон: чередование шагов распознавателей заканчивается.
- Анализ программ — программа о программе: полнота обещаний инструментов запрещена.

Вопросы для самопроверки

Вопросы для самопроверки — о проблеме остановки

1. Воспроизведите схему диагонализации Кантора и укажите, где в ней живёт инверсия.
2. Проведите доказательство неразрешимости: что делает D на входе $\langle D \rangle$ в обоих случаях?
3. Почему HALT распознаваема, а её дополнение — нет?
4. Сформулируйте критерий разрешимости и объясните, почему чередование шагов двух распознавателей всегда заканчивается.
5. Определите сведение $A \leq_m B$ и объясните, в какую сторону его строить для доказательства неразрешимости.
6. Опишите обёртку в сведении от HALT к задаче «программа печатает 42».
7. Опишите сведение от HALT к проблеме пустоты языка.

Что почитать

- m26 — «Разрешимость», «Проблема остановки», «Сведения».